

AI File Intelligence Bot - End-to-End Build Steps

1. SYSTEM OVERVIEW

Build a full-stack AI system using React (frontend), Python FastAPI (backend), and RAG vector database for document intelligence and chatbot capabilities.

2. FRONTEND (REACT)

- File upload UI
- Chat interface
- Response viewer
- Status tracker for uploaded files

3. BACKEND (PYTHON FASTAPI)

- Accept file uploads
- Detect file type (PDF, TXT, Image, Video)
- Route to correct processing pipeline

4. FILE PROCESSING LAYER

- PDF: PyMuPDF / PyPDF2
- Images: pytesseract OCR
- DOCX: python-docx
- Video: Whisper speech-to-text

5. TEXT EXTRACTION PIPELINE

- Extract raw text
- Clean and normalize text
- Detect language (English/Arabic/multilingual)

6. SEMANTIC CHUNKING

- Split text into 500–1000 token chunks
- Add overlap (100 tokens)
- Preserve semantic meaning

7. EMBEDDING GENERATION

- Use sentence-transformers or OpenAI embeddings
- Convert chunks into vector embeddings

8. VECTOR DATABASE (RAG MEMORY)

- Store embeddings in FAISS/ChromaDB
- Attach metadata (file name, chunk id, language)

9. RETRIEVAL SYSTEM

- Convert user query into embedding
- Perform similarity search (top-k chunks)
- Retrieve relevant context

10. LLM RESPONSE ENGINE

- Combine retrieved chunks + user query
- Use prompt engineering for reasoning-based answers
- Output only grounded responses

11. CHATBOT FLOW

User query → embedding → retrieval → LLM → response → frontend display

12. MEMORY SYSTEM

- Short-term: session context
- Long-term: vector DB knowledge base

13. OUTPUT RULES

- Always respond in English
- Use only retrieved context
- Avoid hallucinations

14. TECH STACK

Frontend: React, TailwindCSS

Backend: FastAPI, Python

AI: Sentence Transformers / OpenAI

DB: FAISS / ChromaDB

15. OPTIONAL ADVANCED FEATURES

- Multi-document RAG
- Auto summarization
- Voice input/output
- Document comparison engine